

System Requirements Document (SRD)

Project: AI-Powered Used Car Price Estimation & Recommendation System

1. Project Overview

The car market is highly unstable due to factors like imports and fuel prices. Estimating the value of a used car is difficult because, unlike new cars with fixed prices, used cars require an individual appraisal. This project aims to estimate the selling price of used cars based on features like age, mileage, fuel type, brand, and transmission.

This project has two main parts:

- **Web Application:** A platform where users enter car details and see price estimates.
- **AI Engine:** A regression model trained on historical data to predict fair market prices.

2. System Requirements

2.1 Functional Requirements (FR)

Search & Filtering Features:

- **FR-1.1 Brand Selection:** Users can choose or search for a specific car brand (Make) from a list.
- **FR-1.2 Model Filtering:** Users can filter results based on the specific car model of the chosen brand.
- **FR-1.3 Year Selection:** Users can filter cars by their manufacturing year (e.g., 2015 - 2024).
- **FR-1.4 Price Range Filter:** Users can set a minimum and maximum price for their search.
- **FR-1.5 Transmission Type:** Users can filter by transmission type (Manual, Automatic, Semi-Auto).
- **FR-1.6 Mileage Filtering:** Users can filter based on how far the car has traveled (Mileage).
- **FR-1.7 Fuel Type Selection:** Users can filter by fuel category (Petrol, Diesel, Hybrid, Electric).
- **FR-1.8 Annual Tax Filter:** Users can search based on the annual road tax cost.
- **FR-1.9 Fuel Consumption (MPG):** Users can filter based on fuel efficiency (Miles Per Gallon).
- **FR-1.10 Engine Size:** Users can filter by engine capacity (e.g., 1.2L, 2.0L).

Core AI Functions:

- **FR-2 AI Price Prediction:** The system must provide a price estimate based on the car details entered by the user.
- **FR-3 Data Validation:** The system checks all numerical inputs to ensure they are realistic before processing.

- **FR-4 Display Results:** The system shows the predicted price along with a comparison to similar market deals.

3. SDLC Phase: Analysis (Data Research & Selection)

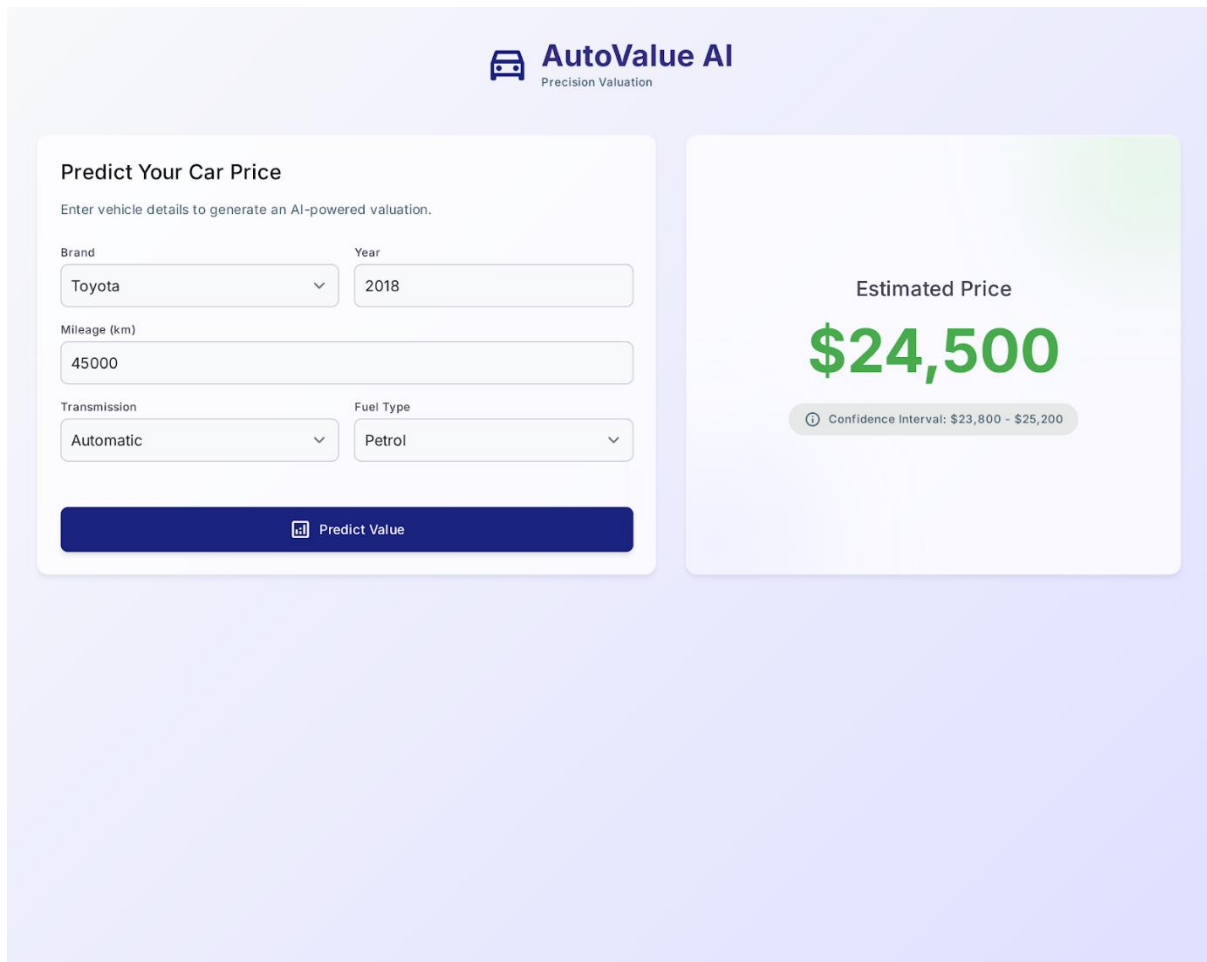
During the analysis phase, three different datasets were researched and compared to find the best one for AI training:

1. **Vehicle Dataset (Kaggle - CarDekho):** [Link](#)
2. **Used Cars Dataset (Craigslist):** [Link](#)
3. **UK Used Cars (Ford and Mercedes):** [Link](#)

Selection Criteria: We chose the **UK Used Cars Dataset** because it is well-organized, the columns match perfectly across files, and it contains over 99,000 records.

4. SDLC Phase: Design (How will it work?)

- **UI/UX:** A simple dashboard with filters for all parameters (FR-1.1 to FR-1.10).



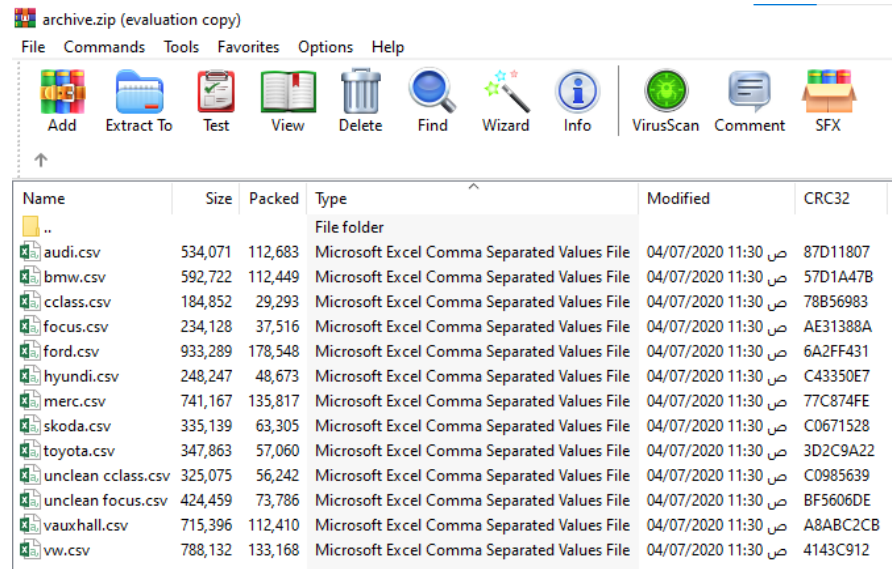
The screenshot displays the 'AutoValue AI' dashboard, which is designed for car price prediction. The dashboard features a light purple background and a central white area with two main sections. On the left, there is a form titled 'Predict Your Car Price' with the instruction 'Enter vehicle details to generate an AI-powered valuation.' The form includes five input fields: 'Brand' (a dropdown menu showing 'Toyota'), 'Year' (a text input showing '2018'), 'Mileage (km)' (a text input showing '45000'), 'Transmission' (a dropdown menu showing 'Automatic'), and 'Fuel Type' (a dropdown menu showing 'Petrol'). Below these fields is a dark blue button with a white bar chart icon and the text 'Predict Value'. On the right, there is a white box with a green gradient header that says 'Estimated Price'. Below this, the price '\$24,500' is displayed in large green font. At the bottom of this box, a grey bar shows the 'Confidence Interval: \$23,800 - \$25,200' with a small information icon on the left.

- **AI Logic:** Using a Regression algorithm to handle multiple inputs and predict the price.

5. SDLC Phase: Implementation (Building the system)

5.1 Data Acquisition & Raw Dataset

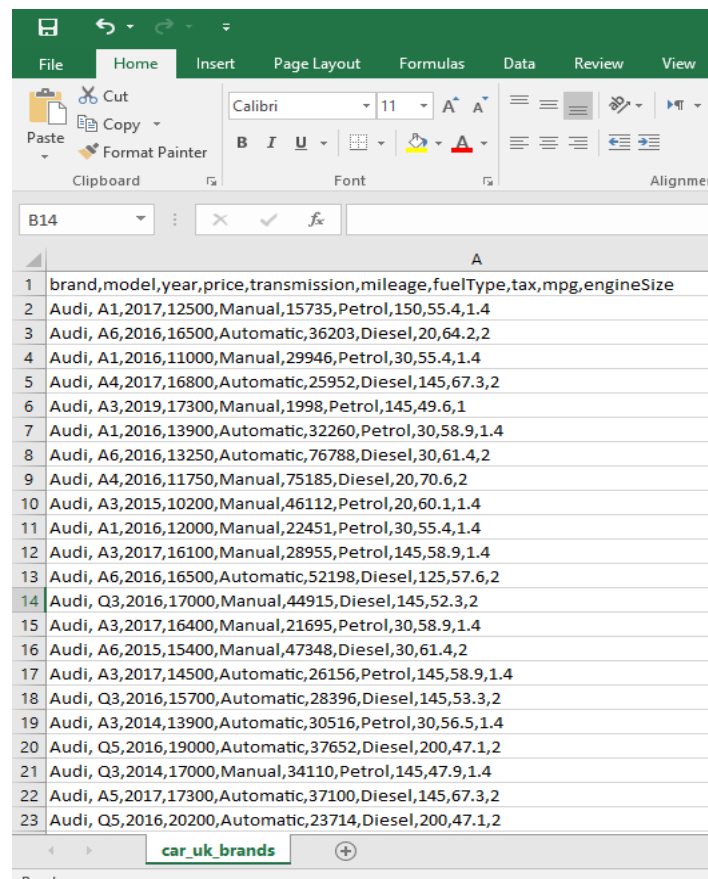
These are the separate CSV files for each brand before they are combined and cleaned.



Name	Size	Packed	Type	Modified	CRC32
..			File folder		
audi.csv	534,071	112,683	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	87D11807
bmw.csv	592,722	112,449	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	57D1A47B
cclass.csv	184,852	29,293	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	78B56983
focus.csv	234,128	37,516	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	AE31388A
ford.csv	933,289	178,548	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	6A2FF431
hyundi.csv	248,247	48,673	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	C43350E7
merc.csv	741,167	135,817	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	77C874FE
skoda.csv	335,139	63,305	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	C0671528
toyota.csv	347,863	57,060	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	3D2C9A22
unclean cclass.csv	325,075	56,242	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	C0985639
unclean focus.csv	424,459	73,786	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	BF5606DE
vauxhall.csv	715,396	112,410	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	A8ABC2CB
vw.csv	788,132	133,168	Microsoft Excel Comma Separated Values File	04/07/2020 11:30 ص	4143C912

5.2 Data Transformation & Cleaned Dataset

The data after being merged into a single file named car_uk_brands.csv.



	brand,model,year,price,transmission,mileage,fuelType,tax,mpg,engineSize
1	Audi, A1,2017,12500,Manual,15735,Petrol,150,55.4,1.4
2	Audi, A6,2016,16500,Automatic,36203,Diesel,20,64.2,2
3	Audi, A1,2016,11000,Manual,29946,Petrol,30,55.4,1.4
4	Audi, A4,2017,16800,Automatic,25952,Diesel,145,67.3,2
5	Audi, A3,2019,17300,Manual,1998,Petrol,145,49.6,1
6	Audi, A1,2016,13900,Automatic,32260,Petrol,30,58.9,1.4
7	Audi, A6,2016,13250,Automatic,76788,Diesel,30,61.4,2
8	Audi, A4,2016,11750,Manual,75185,Diesel,20,70.6,2
9	Audi, A3,2015,10200,Manual,46112,Petrol,20,60.1,1.4
10	Audi, A1,2016,12000,Manual,22451,Petrol,30,55.4,1.4
11	Audi, A3,2017,16100,Manual,28955,Petrol,145,58.9,1.4
12	Audi, A6,2016,16500,Automatic,52198,Diesel,125,57.6,2
13	Audi, Q3,2016,17000,Manual,44915,Diesel,145,52.3,2
14	Audi, A3,2017,16400,Manual,21695,Petrol,30,58.9,1.4
15	Audi, A6,2015,15400,Manual,47348,Diesel,30,61.4,2
16	Audi, A3,2017,14500,Automatic,26156,Petrol,145,58.9,1.4
17	Audi, Q3,2016,15700,Automatic,28396,Diesel,145,53.3,2
18	Audi, A3,2014,13900,Automatic,30516,Petrol,30,56.5,1.4
19	Audi, Q5,2016,19000,Automatic,37652,Diesel,200,47.1,2
20	Audi, Q3,2014,17000,Manual,34110,Petrol,145,47.9,1.4
21	Audi, A5,2017,17300,Automatic,37100,Diesel,145,67.3,2
22	Audi, Q5,2016,20200,Automatic,23714,Diesel,200,47.1,2

5.3 Model Building & Data Verification

loads the merged car dataset and displays the first five rows to verify the columns and data structure.

```
import pandas as pd
df = pd.read_csv('car_uk_brands.csv')
display(df.head())
```

```
print("Data Missed in Each Column:")
print(df.isnull().sum())
print("-" * 50)
print("Num Of Duplicated Rows:", df.duplicated().sum())
print("-" * 50)
df.info()
```

cell output

```
Data Missed in Each Column:
model      0
year       0
price      0
transmission  0
mileage    0
fuelType   0
tax        0
mpg        0
engineSize  0
brand      0
dtype: int64
-----
Num Of Duplicated Rows: 1475
-----
<class 'pandas.DataFrame'>
RangeIndex: 99187 entries, 0 to 99186
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   model      99187 non-null  str
1   year       99187 non-null  int64
2   price      99187 non-null  int64
3   transmission 99187 non-null  str
4   mileage    99187 non-null  int64
...
8   engineSize 99187 non-null  float64
9   brand      99187 non-null  str
dtypes: float64(2), int64(4), str(4)
memory usage: 7.6 MB
```

5.4 Data Cleaning (Duplicates & Outliers)

After removing duplicates, we remove "Outliers" (unrealistic data) to ensure accuracy.

1. Handling Duplicates & Outliers:

```
df = df.drop_duplicates()
print("Num Of Duplicated Rows After Removing Duplicates:", df.duplicated().sum())
print("-" * 50)
display(df.describe())
```

cell output

```
Num Of Duplicated Rows After Removing Duplicates: 0
```

```
-----
```

```
df = df[df['year'] <= 2026]
df = df[df['engineSize'] > 0]
df = df[df['mpg'] <= 200]
```

```
print("Num Of Rows After Removing Outliers:", len(df))
```

cell output

```
Num Of Rows After Removing Outliers: 97397
```

2. This code generates visual charts to analyze how car prices are distributed across different brands and how they decrease as mileage increases.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

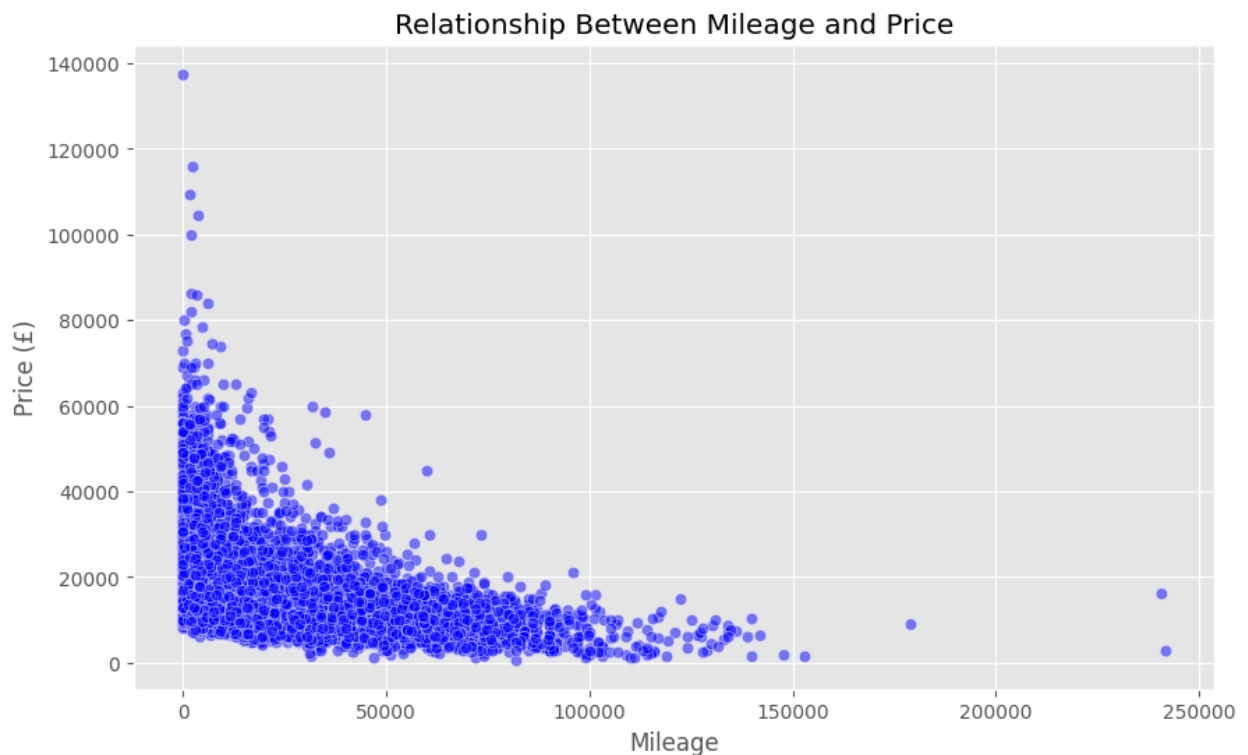
# Settings of shapes
plt.style.use('ggplot')

# 1. (Price Distribution across Brands)
plt.figure(figsize=(12, 6))
sns.boxplot(x='brand', y='price', data=df)
plt.title('Price Distribution Across Different Car Brands')
plt.xlabel('Car Brand')
plt.ylabel('Price (£)')
plt.xticks(rotation=45)
plt.show()

print("-" * 50)

# 2. (Mileage vs. Price Relationship) sample
plt.figure(figsize=(10, 6))
sns.scatterplot(x='mileage', y='price', data=df.sample(10000, random_state=42), alpha=0.5, color='blue')
plt.title('Relationship Between Mileage and Price')
plt.xlabel('Mileage')
plt.ylabel('Price (£)')
plt.show()
```

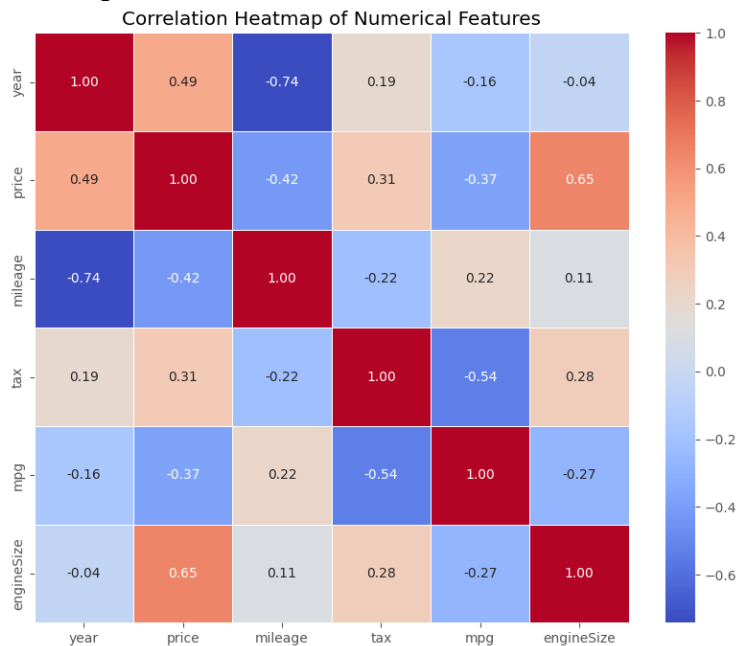
cell output



3. This code creates a heatmap to visualize the mathematical relationships and correlations between numerical features like price, year, and mileage.

```
# Correlation Heatmap of Numerical Features
plt.figure(figsize=(10, 8))
numerical_cols = df.select_dtypes(include=[np.number])
sns.heatmap(numerical_cols.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

cell output



4. This code visualizes the upward trend of average car prices over time, specifically showing how prices change every five years from 1995 to 2020.

```
# متوسط سعر كل 5 سنة من اول سنة 1995 علشان مفيش غير عربية واحدة قبل كده
plt.figure(figsize=(12, 6))
df_trend = df[df['year'] >= 1995]
avg_price_year = df_trend.groupby('year')['price'].mean().reset_index()
sns.lineplot(x='year', y='price', data=avg_price_year, marker='o', color='green', linewidth=2)

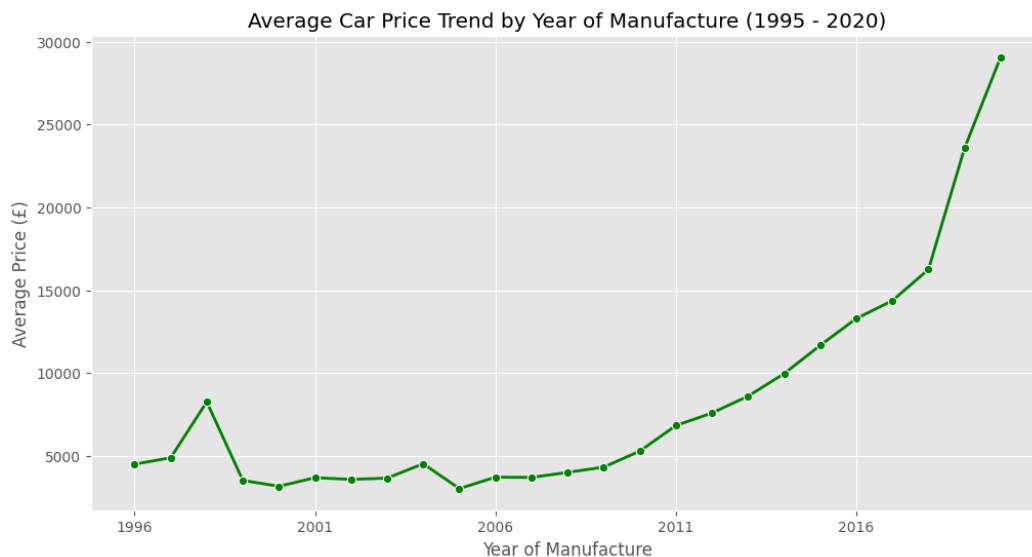
plt.title('Average Car Price Trend by Year of Manufacture (1995 - 2020)')
plt.xlabel('Year of Manufacture')
plt.ylabel('Average Price (£)')

# كل 5 سنين
min_year = int(min(avg_price_year['year']))
max_year = int(max(avg_price_year['year']))

plt.xticks(np.arange(min_year, max_year + 1, 5))

plt.show()
```

cell output



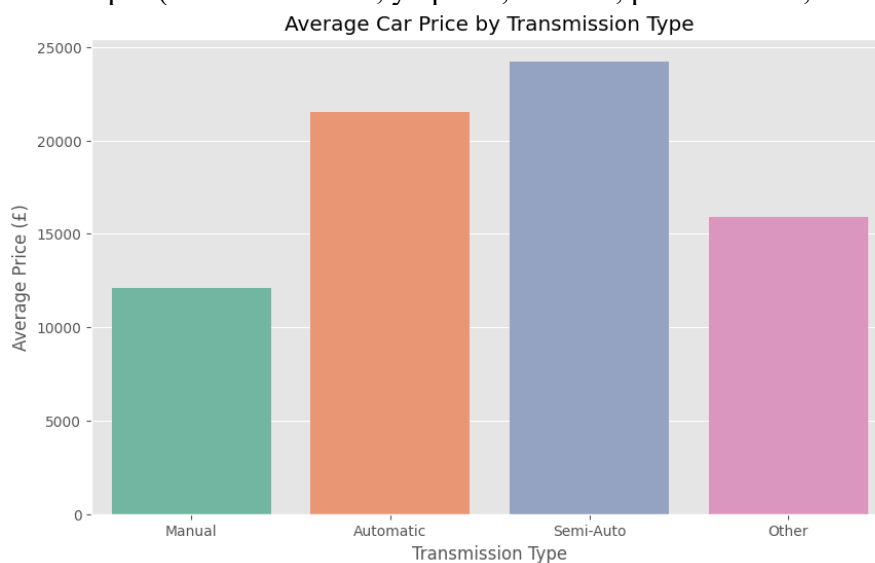
5. This code displays a bar chart comparing the average selling prices of cars based on their transmission types to identify which category holds the highest market value.

```
# متوسط السعر حسب نوع القياس
plt.figure(figsize=(10, 6))
sns.barplot(x='transmission', y='price', data=df, palette='Set2', errorbar=None)
plt.title('Average Car Price by Transmission Type')
plt.xlabel('Transmission Type')
plt.ylabel('Average Price (£)')
plt.show()
```

cell output

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='transmission', y='price', data=df, palette='Set2', errorbar=None)
```



6. This code creates a histogram to visualize the distribution of car prices and identify the most common price ranges in the dataset.

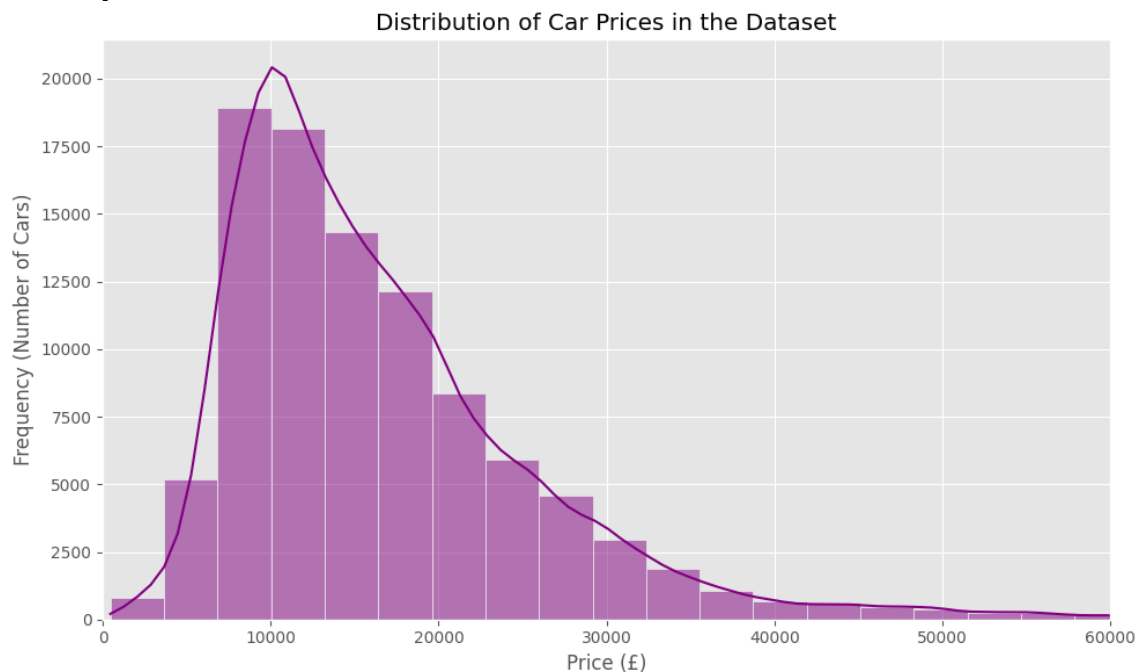
```
# price distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['price'], bins=50, kde=True, color='purple')

plt.title('Distribution of Car Prices in the Dataset')
plt.xlabel('Price (£)')
plt.ylabel('Frequency (Number of Cars)')

plt.xlim(0, 60000)

plt.tight_layout()
plt.show()
```

cell output



7. This code identifies the text-based categories in the dataset and prints the total number of rows and columns before converting them into numerical format for the AI model.

```
categorical_columns = ['brand', 'model', 'transmission', 'fuelType']
print(f"Table Before Encoding: {df.shape[0]} rows, {df.shape[1]} columns")
```

cell output

Table Before Encoding: 97397 rows, 10 columns

8. This code transforms text-based categories into numerical columns using one-hot encoding to prepare the data for machine learning training.

```
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)
print("-" * 50)
```

```
print(f'table After Encoding: {df_encoded.shape[0]} rows, {df_encoded.shape[1]} columns')
display(df_encoded.head())
```

cell output

table After Encoding: 97397 rows, 213 columns

Model train and test

1. This code splits the data into training and testing sets and trains a Random Forest Regressor to learn the patterns for predicting car prices.

💡 # Model Train and Test

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
X = df_encoded.drop('price', axis=1)
y = df_encoded['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print("Model Training")
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)
```

cell output

Model Training

2. This code evaluates the trained model's performance on the test set by calculating its accuracy and the average prediction error in pounds.

```
# test
predictions = rf_model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)
print(f"First Result (Random Forest): {r2 * 100:.2f}%")
print(f"Model Accuracy : {r2 * 100:.2f}%")
print(f"Mean Absolute Error: £{mae:.2f}")
```

cell output

First Result (Random Forest): 95.67%

Model Accuracy : 95.67%

Mean Absolute Error: £1163.17

3. This code extracts the most influential factors in determining the price of a car from the Random Forest model and sorts them in descending order to display the top 10 features in an explanatory graph.

```
# Feature Importance
importances = rf_model.feature_importances_
feature_imp_df = pd.DataFrame({'Feature': X_train.columns, 'Importance': importances})
feature_imp_df = feature_imp_df.sort_values(by='Importance', ascending=False).head(10)

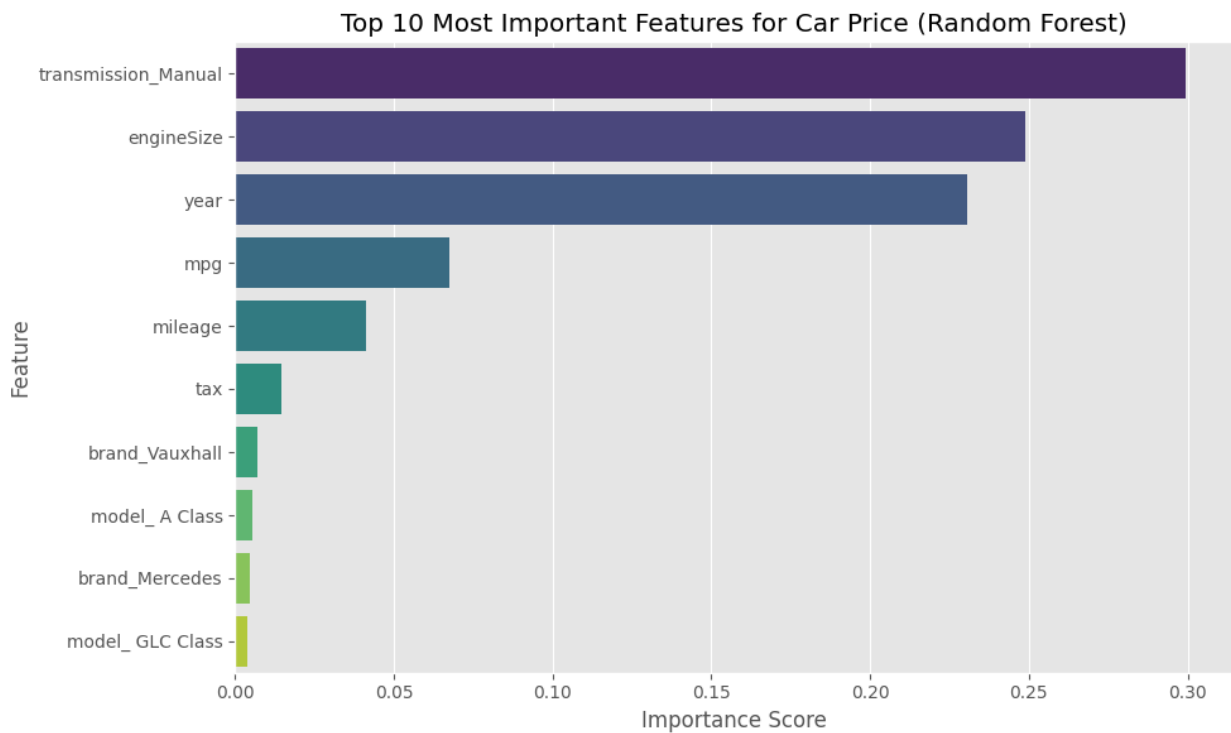
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_imp_df, palette='viridis')
plt.title('Top 10 Most Important Features for Car Price (Random Forest)')
plt.xlabel('Importance Score')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

cell output

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='Importance', y='Feature', data=feature_imp_df, palette='viridis')
```



4. This code trains an alternative machine learning model using the XGBoost algorithm to compare its performance against the previous model and potentially achieve higher prediction accuracy.

```
# New Model For Comparing
```

```
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

```
xgb_model = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
```

```
# Train
xgb_model.fit(X_train, y_train)
```

5. This code evaluates the performance of the XGBoost model by calculating its accuracy and the average prediction error to compare it with the first model.

```
xgb_predictions = xgb_model.predict(X_test)
xgb_mae = mean_absolute_error(y_test, xgb_predictions)
xgb_r2 = r2_score(y_test, xgb_predictions)

print(f"Second Result (XGBoost):")
print(f"Model Accuracy : {xgb_r2 * 100:.2f}%")
print(f"Mean Absolute Error: £±{xgb_mae:.2f}")
```

cell output

Second Result (XGBoost):

Model Accuracy : 93.72%

Mean Absolute Error: £±1590.62

5.5 Model Tuning Results after Implementation

```
# ---- Summary table (baseline vs tuned) ----
results = []
results.append({"model": "RandomForest (baseline)", **baseline_rf})
results.append({"model": "XGBoost (baseline)", **baseline_xgb})

# rf_tuned_metrics exists after running the RF tuning cell above
if "rf_tuned_metrics" in globals():
    results.append({"model": "RandomForest (tuned)", **rf_tuned_metrics})

# xgb_tuned_metrics exists after running the Optuna cell above
if "xgb_tuned_metrics" in globals():
    results.append({"model": "XGBoost (tuned)", **xgb_tuned_metrics})

results_df = pd.DataFrame(results).sort_values(by="rmse")
display(results_df)
```

	model	rmse	mae	r2
3	XGBoost (tuned)	1922.575811	1078.347900	0.962255
2	RandomForest (tuned)	1999.025368	1139.406954	0.959193
0	RandomForest (baseline)	2060.523795	1162.707897	0.956644
1	XGBoost (baseline)	2480.303409	1590.421143	0.937179

6. SDLC Phase: Testing (Checking for Accuracy)

- **Data Integrity Test:** Verified that all duplicates and unrealistic values were removed, leaving 97,397 high-quality records.
- **Accuracy Test:** Testing the AI's price prediction against real samples from the test set.
- **API Testing :** Testing the Fast API that have done .

Input Image test

POST /predict Predict Price

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{  "brand": "Audi",  "model": "A1",  "year": 2017,  "transmission": "Manual",  "mileage": 15735,  "fuelType": "Petrol",  "tax": 150,  "mpg": 55.4,  "engineSize": 1.4}
```

Execute Clear Activate Windows
Go to Settings to activate Windows.

Output Result

Server response	
Code	Details
200	Response body<pre>{ "estimated_price": 12651.49, "currency": "GBP", "model_used": "XGBoost (tuned)"}</pre>

7. SDLC Phase: Deployment & Maintenance

- **Monitoring:** Regularly checking the AI performance and updating the data with new market prices to keep estimates accurate.

Project Success Checklist

1. **Analysis:** Best data source selected (UK Used Cars).
2. **Data Transformation:** Files merged and quality checked.
3. **Data Cleaning:** Duplicates removed and outliers filtered.
4. **AI Training:** Training the model using the right Regression algorithm.
5. **Website Link Deployed :** <https://87a6110f-e579-4e84-8421-8143fba5c3b-00-g5or3x8ct806.worf.replit.dev/>